

# Week 11

## 3. Database Programming: Apache Derby: Java RDBMS

Dr. Andreas S. Maniatis  
Assistant Professor

School of Computer Science  
COMP.2800 – Software Development [2026W]

Wednesday, March 18<sup>th</sup>, 2026 | 14:30 – 15:00 | ER 1118



University of Windsor

# Agenda

- Recap on MySQL
- Install Apache Derby
- Configure Apache Derby
- Use Apache Derby
- Wrap-up



# Part I

## MySQL Recap



## MySQL Recap



# MySQL RDBMS: Recap

- MySQL:
  - Popular Open-Source Relational Database Management System (RDBMS)
  - Utilizes SQL to store and manage data
  - Known for reliability and scalability, it is widely used for web applications and is owned by Oracle Corporation.
  - Key Features:
    - Relational Structure: Organizes data into tables with rows and columns for consistency.
    - SQL & Architecture: Uses Standard Query Language (SQL) within a client-server model, where applications query a central server.
    - ACID Compliance: Features reliable transaction processing, particularly with the InnoDB engine.
    - Open-Source & Multi-Platform: Free under GPL, with support for Linux, Windows, and macOS.
  - Common Use Cases:

As part of the LAMP stack, MySQL is heavily used for:

    - Web & CMS: Powering sites like WordPress, Facebook, and Twitter.
    - E-commerce & SaaS: Managing data for platforms like Shopify and Uber.
    - Enterprise: Used for scalable, mission-critical applications.
- Tutorial on MySQL:
  - SQL
  - Access from Java



## Part II

# Apache Derby





## About Apache Derby



# Apache Derby

- Apache Derby is:

- An open-source RDBMS
- Implemented entirely in Java
- Available under the [Apache License, Version 2.0](#).

☐ Derby has a small footprint

☐ It is based on Java, JDBC and SQL standards

☐ Derby provides an embedded JDBC driver

☐ Derby is easy to install, deploy, and use.

## Derby Retired!

On 2025-10-10, the Derby developers voted to retire the project into a read-only state.

It is still useful to familiarize with a lightweight / light footprint Java RDBMS.





## ○ Installing Apache Derby



## Installing Apache Derby

- The best way to install Apache Derby on your environment is to use the *Getting Started* guide:
  - <https://db.apache.org/derby/docs/10.17/getstart/index.html>
- The instructions in the following slides are a summary of the *Getting Started* guide but may be slightly inconsistent with the latest Apache Derby version.



# Installing Apache Derby

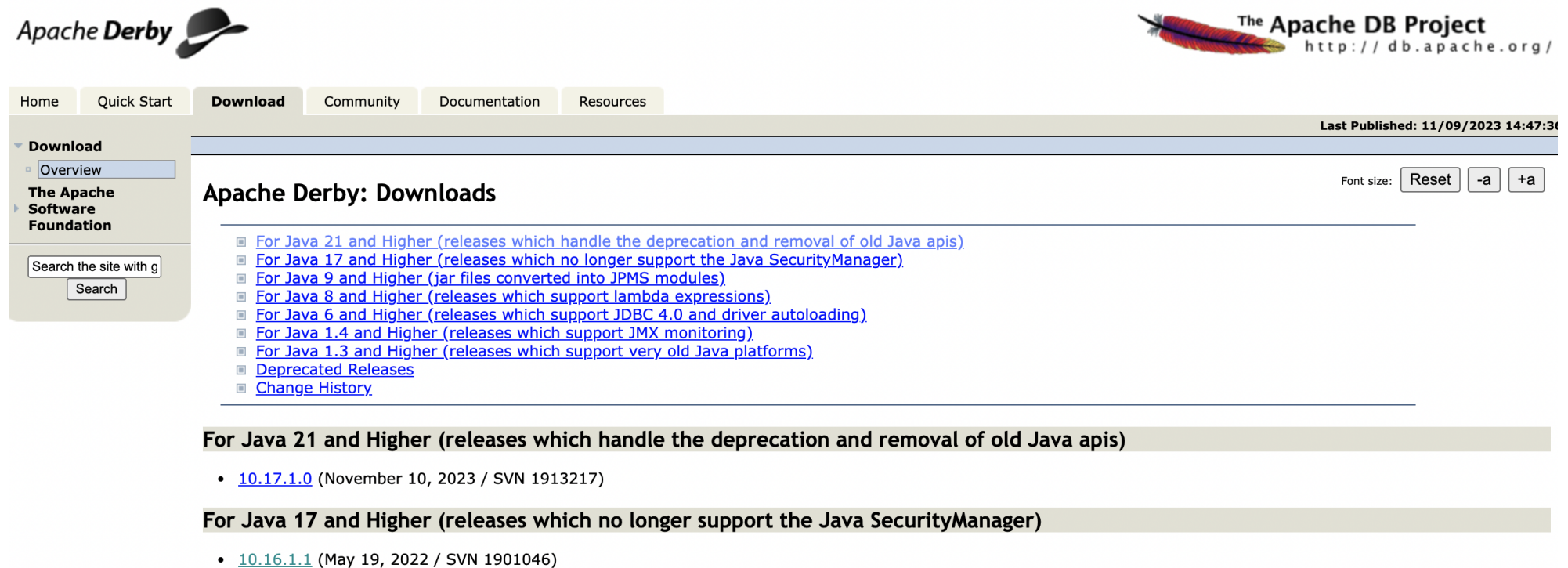
- Find the JDK installation folder:

- 32-bit JDK on Windows
  - C:\Program Files (x86)\Java\jdk1.7.0\_51
- 64-bit JDK on Windows
  - C:\Program Files\Java\jdk1.7.0\_51
- Mac OS X  
/Library/Java/JavaVirtualMachines/jdk1.7.0\_51.jdk/Contents/Home
- Ubuntu Linux  
/usr/lib/jvm/java-7-oracle



# Download Apache Derby

- Download Apache Derby:  
[https://db.apache.org/derby/derby\\_downloads.html](https://db.apache.org/derby/derby_downloads.html)



The screenshot shows the Apache Derby Downloads page. At the top, there's the Apache Derby logo and the Apache DB Project logo with the URL <http://db.apache.org/>. Below the logos is a navigation bar with tabs: Home, Quick Start, Download (selected), Community, Documentation, and Resources. On the left side, there's a sidebar with a search box and a list of links: Download, Overview, The Apache Software Foundation, and a search button. The main content area is titled "Apache Derby: Downloads" and lists several download links for different Java versions. The first link is "For Java 21 and Higher (releases which handle the deprecation and removal of old Java apis)", which is expanded to show a list of versions: 10.17.1.0 (November 10, 2023 / SVN 1913217). The second link is "For Java 17 and Higher (releases which no longer support the Java SecurityManager)", which is also expanded to show a list of versions: 10.16.1.1 (May 19, 2022 / SVN 1901046). The page also includes a "Last Published" timestamp of 11/09/2023 14:47:30 and a font size control.

Apache Derby

The Apache DB Project  
<http://db.apache.org/>

Home Quick Start **Download** Community Documentation Resources

Last Published: 11/09/2023 14:47:30

Font size: Reset -a +a

### Apache Derby: Downloads

- For Java 21 and Higher (releases which handle the deprecation and removal of old Java apis)
  - 10.17.1.0 (November 10, 2023 / SVN 1913217)
- For Java 17 and Higher (releases which no longer support the Java SecurityManager)
  - 10.16.1.1 (May 19, 2022 / SVN 1901046)

# Download Apache Derby

- Select the distribution of Apache Derby to download

## Apache Derby 10.14.2.0 Release

Font size: |

- ▣ [Distributions](#)
- ▣ [Release Notes for Apache Derby 10.14.2.0](#)
  - ▣ [Overview](#)
  - ▣ [New Features](#)
  - ▣ [Bug Fixes](#)
  - ▣ [Issues](#)
  - ▣ [Build Environment](#)
  - ▣ [Verifying Releases](#)

### Distributions

Use the links below to download a distribution of Apache Derby. You should **always** [verify the integrity](#) of distribution files downloaded from a mirror.

There are four different distributions:

- bin distribution - contains the documentation, javadoc, and jar files for Derby.
- lib distribution - contains only the jar files for Derby.
- lib-debug distribution - contains jar files for Derby with source line numbers.
- src distribution - contains the Derby source tree at the point which the binaries were built.

[db-derby-10.14.2.0-bin.zip](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄  
[db-derby-10.14.2.0-bin.tar.gz](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄

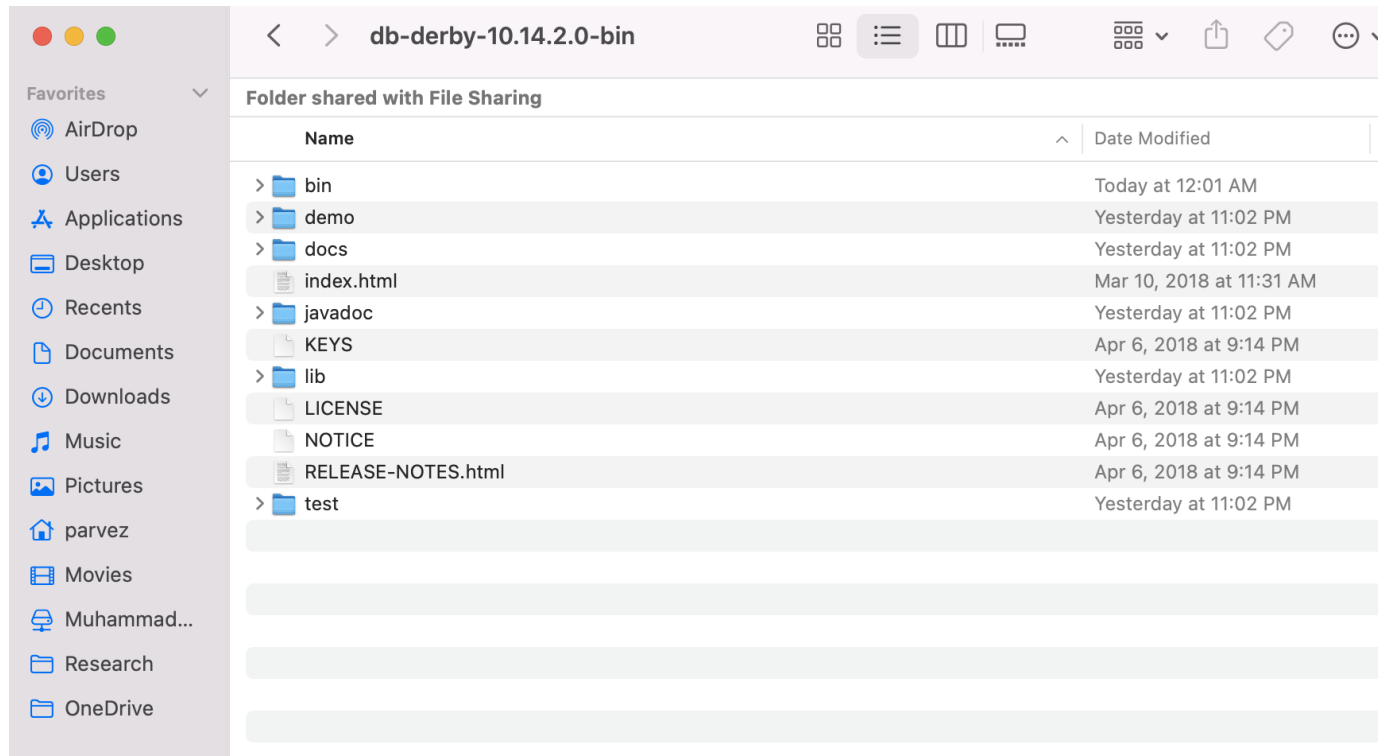
[db-derby-10.14.2.0-lib.zip](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄  
[db-derby-10.14.2.0-lib.tar.gz](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄

[db-derby-10.14.2.0-lib-debug.zip](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄  
[db-derby-10.14.2.0-lib-debug.tar.gz](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄

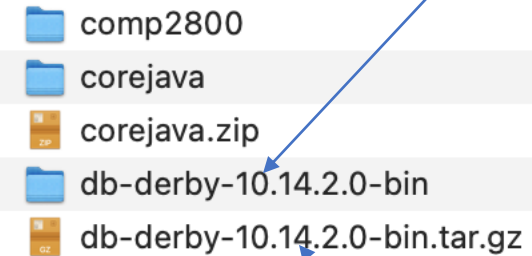
[db-derby-10.14.2.0-src.zip](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄  
[db-derby-10.14.2.0-src.tar.gz](#) ⇄ [PGP](#) ⇄ [MD5](#) ⇄ (Note that, due to long filenames, you will need gnu tar to unravel this tarball.)



# Installing Apache Derby



Extracted Folder



Downloaded distribution

## Installing Apache Derby

- Set the JAVA\_HOME environment variable to the root location of the JDK installation directory:

```
set JAVA_HOME = /Library/Java/JavaVirtualMachines/jdk-18.0.2.jdk/Contents/Home
```



## ● Installing Apache Derby

- • Set the PATH environment variable to include the JDK bin directory.
- The PATH variable tells the operating system where to find the java interpreter and the javac compiler.

```
set PATH=$JAVA_HOME:$PATH
```





## Installing Apache Derby

- Use the `java -version` command, as shown below, to verify the installed release: `java -version`

```
2. bash
Last login: Tue Mar 19 23:11:55 on ttys001

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
(base) MacBook-Pro-999:~ parvez$ java -version
java version "18.0.2" 2022-07-19
Java(TM) SE Runtime Environment (build 18.0.2+9-61)
Java HotSpot(TM) 64-Bit Server VM (build 18.0.2+9-61, mixed mode, sharing)
(base) MacBook-Pro-999:~ parvez$
```

## Installing Apache Derby

- Download the binary Apache Derby distribution from the Derby web site at:  
[http://db.apache.org/derby/derby\\_downloads.html](http://db.apache.org/derby/derby_downloads.html)

| Operating System     | Download File                 |
|----------------------|-------------------------------|
| Windows              | db-derby-10.17.1.0-bin.zip    |
| UNIX, Linux, and Mac | db-derby-10.17.1.0-bin.tar.gz |



## ● Installing Apache Derby

- • Set the DERBY\_INSTALL variable to the location where you installed Derby:

```
set DERBY_INSTALL=/Users/parvez/Desktop/db-derby-10.14.2.0-bin
```



## Installing Apache Derby

- To use Derby in its embedded mode set your CLASSPATH to include the jar files listed below:
  - `derby.jar`: Contains the Derby engine and the Derby Embedded JDBC driver
  - `derbytools.jar`: Optional, provides the `ij` tool that is used by a couple of sections in this tutorial

set

```
CLASSPATH=$DERBY_INSTALL/lib/derby.jar:$DERBY_INSTALL/lib/derbytools.jar:$DERBY_INSTALL/lib/derbyoptionaltools.jar:$DERBY_INSTALL/lib/derbyshared.jar:.
```



## ● Installing Apache Derby

- • Change directory now into the DERBY\_INSTALL/bin directory

```
cd $DERBY_INSTALL/bin
```

```
. setEmbeddedCP
```



## ● Installing Apache Derby

- • Run the `sysinfo` command, as shown below, to output Derby system information:

```
java org.apache.derby.tools.sysinfo
```

# Installing Apache Derby

```
(base) MacBook-Pro-999:bin parvez$ java org.apache.derby.tools.sysinfo
----- Java Information -----
Java Version:      18.0.2
Java Vendor:       Oracle Corporation
Java home:         /Library/Java/JavaVirtualMachines/jdk-18.0.2.jdk/Contents/Home
Java classpath:    /Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derby.jar:/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbytools.jar:/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbyoptionaltools.jar:/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derby.jar:/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbytools.jar:/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbyoptionaltools.jar:/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbyshared.jar:.
OS name:           Mac OS X
OS architecture:  x86_64
OS version:        12.4
Java user name:    parvez
Java user home:    /Users/parvez
Java user dir:     /Users/parvez/Desktop/db-derby-10.14.2.0-bin/bin
java.specification.name: Java Platform API Specification
java.specification.version: 18
java.runtime.version: 18.0.2+9-61
----- Derby Information -----
[/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derby.jar] 10.14.2.0 - (1828579)
[/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbytools.jar] 10.14.2.0 - (1828579)
----- Derby Information -----
[/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derby.jar] 10.14.2.0 - (1828579)
[/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbytools.jar] 10.14.2.0 - (1828579)
[/Users/parvez/Desktop/db-derby-10.14.2.0-bin/lib/derbyoptionaltools.jar] 10.14.2.0 - (1828579)
-----
----- Locale Information -----
-----
```

shown below, to output Derby

`derby.tools.sysinfo`



## ○ Installing Apache Derby: **ij**





## Installing Apache Derby: **ij**

- **ij** is an interactive SQL scripting tool that comes with Derby
- Connect to **ij**

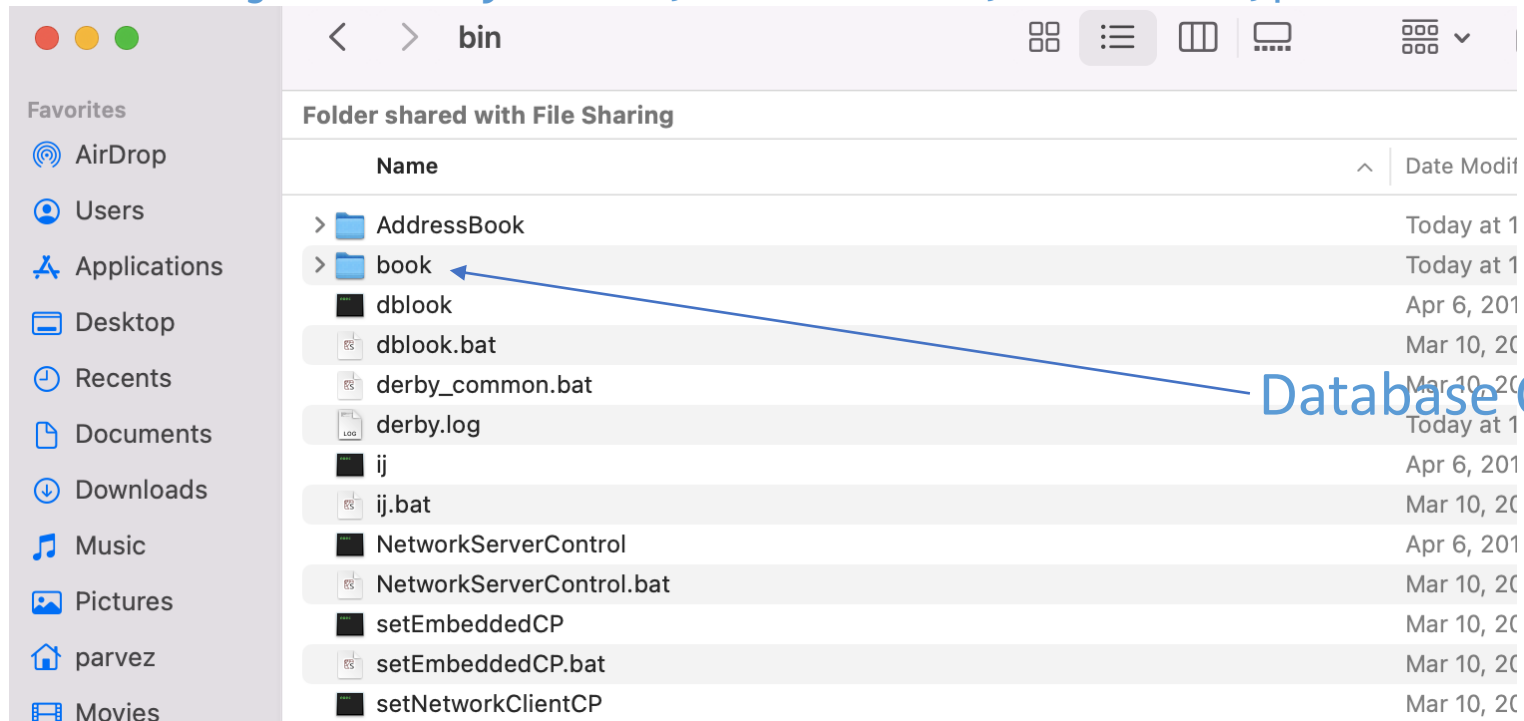
```
(base) MacBook-Pro-999:Desktop parvez$ cd db-derby-10.14.2.0-bin
(base) MacBook-Pro-999:db-derby-10.14.2.0-bin parvez$ cd bin
(base) MacBook-Pro-999:bin parvez$ ls
AddressBook                               setEmbeddedCP.bat
NetworkServerControl                     setNetworkClientCP
NetworkServerControl.bat                 setNetworkClientCP.bat
book                                     setNetworkServerCP
dblook                                  setNetworkServerCP.bat
dblook.bat                             startNetworkServer
derby.log                             startNetworkServer.bat
derby_common.bat                       stopNetworkServer
ij                                      stopNetworkServer.bat
ij.bat                                 sysinfo
setEmbeddedCP                          sysinfo.bat
(base) MacBook-Pro-999:bin parvez$ ./ij
ij version 10.14
ij> 
```



# Installing Apache Derby : ij

- At the `ij>` prompt type

```
connect 'jdbc:derby:books;create=true;user=root;password=secret';
```



Database Created

## Installing Apache Derby : `ij`

- To create the database table and insert sample data in it, we've provided the file `address.sql` in this example's directory. To execute this SQL script, type:

```
run 'books.sql';
```

```
ij> select*from authors;
AUTHORID  |FIRSTNAME      |LASTNAME
-----|-----|-----
1         |Paul           |Deitel
2         |Harvey         |Deitel
3         |Abbey          |Deitel
4         |Dan            |Quirk
5         |Michael        |Morgano

5 rows selected
```

# Part III

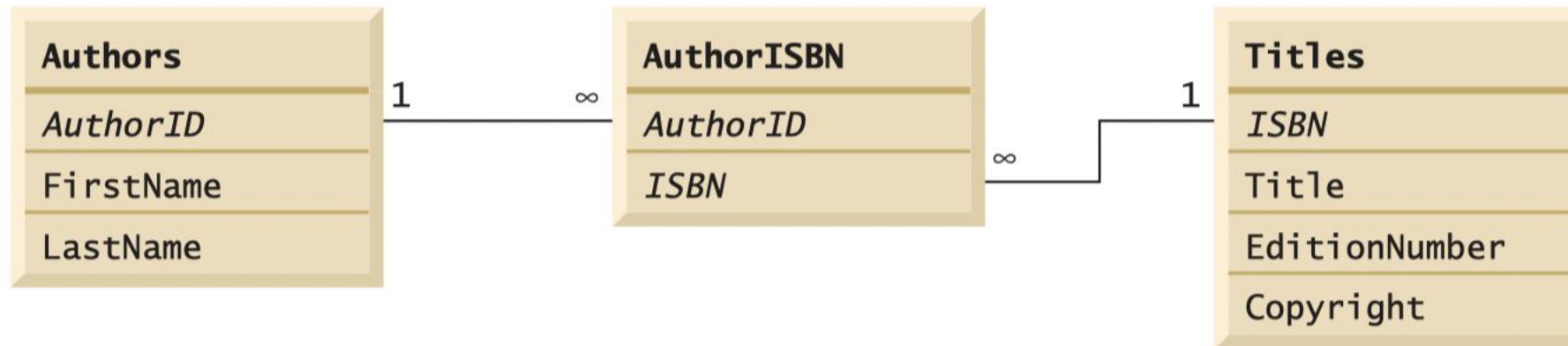
## Working with Databases in Apache Derby



## ○ The Book Database in Apache Derby



# Book Database



## ○ Manipulating Databases with JDBC

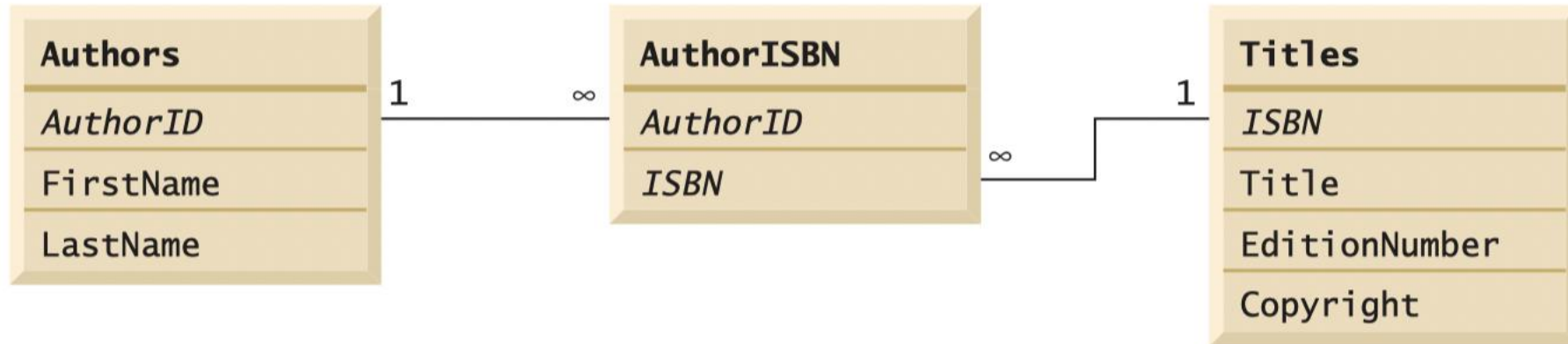


# Manipulating Databases with JDBC

| SQL keyword | Description                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------|
| SELECT      | Retrieves data from one or more tables.                                                                                         |
| FROM        | Tables involved in the query. Required in every SELECT.                                                                         |
| WHERE       | Criteria for selection that determine the rows to be retrieved, deleted or updated. Optional in a SQL query or a SQL statement. |
| GROUP BY    | Criteria for grouping rows. Optional in a SELECT query.                                                                         |
| ORDER BY    | Criteria for ordering rows. Optional in a SELECT query.                                                                         |
| INNER JOIN  | Merge rows from multiple tables.                                                                                                |
| INSERT      | Insert rows into a specified table.                                                                                             |
| UPDATE      | Update rows in a specified table.                                                                                               |
| DELETE      | Delete rows from a specified table.                                                                                             |



# Manipulating Databases with JDBC



```
SELECT AuthorID, FirstName, LastName
FROM Authors
WHERE LastName LIKE '_o%'
```

| AuthorID | FirstName | LastName |
|----------|-----------|----------|
| 5        | Michael   | Morgano  |

# Manipulating Databases with JDBC



```
SELECT AuthorID, FirstName, LastName
FROM Authors
ORDER BY LastName ASC
```

| AuthorID | FirstName | LastName |
|----------|-----------|----------|
| 1        | Paul      | Deitel   |
| 2        | Harvey    | Deitel   |
| 3        | Abbey     | Deitel   |
| 5        | Michael   | Morgano  |
| 4        | Dan       | Quirk    |

# Manipulating Databases with JDBC

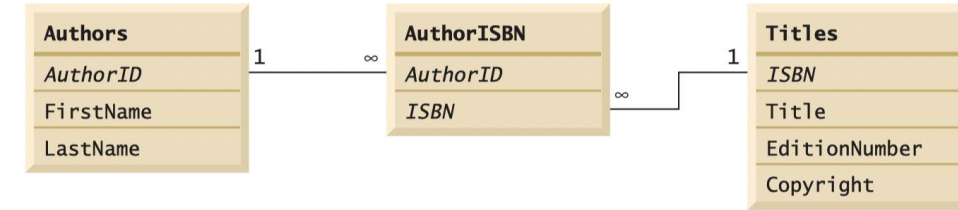
```
SELECT AuthorID, FirstName, LastName  
FROM Authors  
ORDER BY LastName ASC
```

| AuthorID | FirstName | LastName |
|----------|-----------|----------|
| 1        | Paul      | Deitel   |
| 2        | Harvey    | Deitel   |
| 3        | Abbey     | Deitel   |
| 5        | Michael   | Morgano  |
| 4        | Dan       | Quirk    |

```
SELECT AuthorID, FirstName, LastName  
FROM Authors  
ORDER BY LastName DESC
```



# Manipulating Databases with JDBC



```
SELECT ISBN, Title, EditionNumber, Copyright
FROM Titles
WHERE Title LIKE '%How to Program'
ORDER BY Title ASC
```

| ISBN       | Title                                    | EditionNumber | Copyright |
|------------|------------------------------------------|---------------|-----------|
| 0133764036 | Android How to Program                   | 2             | 2015      |
| 013299044X | C How to Program                         | 7             | 2013      |
| 0133378713 | C++ How to Program                       | 9             | 2014      |
| 0132151006 | Internet & World Wide Web How to Program | 5             | 2012      |
| 0133807800 | Java How to Program                      | 10            | 2015      |
| 0133406954 | Visual Basic 2012 How to Program         | 6             | 2014      |
| 0133379337 | Visual C# 2012 How to Program            | 5             | 2014      |
| 0136151574 | Visual C++ 2008 How to Program           | 2             | 2008      |

# Manipulating Databases with JDBC

- Common SQL Data Types

| Data Types                                                     | Description                                                                             |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| INTEGER or INT                                                 | Typically, a 32-bit integer                                                             |
| SMALLINT                                                       | Typically, a 16-bit integer                                                             |
| NUMERIC ( $m, n$ ),<br>DECIMAL ( $m, n$ ) or<br>DEC ( $m, n$ ) | Fixed-point decimal number with $m$ total digits and $n$ digits after the decimal point |
| FLOAT ( $n$ )                                                  | A floating-point number with $n$ binary digits of precision                             |
| REAL                                                           | Typically, a 32-bit floating-point number                                               |
| DOUBLE                                                         | Typically, a 64-bit floating-point number                                               |
| CHARACTER ( $n$ ) or<br>CHAR ( $n$ )                           | Fixed-length string of length $n$                                                       |
| VARCHAR ( $n$ )                                                | Variable-length strings of maximum length $n$                                           |
| BOOLEAN                                                        | A Boolean value                                                                         |
| DATE                                                           | Calendar date, implementation-dependent                                                 |
| TIME                                                           | Time of day, implementation-dependent                                                   |
| TIMESTAMP                                                      | Date and time of day, implementation-dependent                                          |

# Part IV

## Hands-on Practice with Apache Derby



## ○ Prepared SQL Statements for Apache Derby



## Prepared Statements

```
SELECT Books.Price, Books.Title  
FROM Books, Publishers  
WHERE Books.Publisher_Id = Publishers.Publisher_Id  
AND Publishers.Name = the name from the list box
```





## Prepared Statements

- Instead of building a separate query statement every time the user launches such a query, we can prepare a query with a host variable and use it many times, each time filling in a different string for the variable.

```
SELECT Books.Price, Books.Title  
FROM Books, Publishers  
WHERE Books.Publisher_Id = Publishers.Publisher_Id  
AND Publishers.Name = the name from the list box
```



## Prepared Statements

- Instead of building a separate query statement every time the user launches such a query, we can prepare a query with a host variable and use it many times, each time filling in a different string for the variable.

```
SELECT Books.Price, Books.Title  
FROM Books, Publishers  
WHERE Books.Publisher_Id = Publishers.Publisher_Id  
AND Publishers.Name = the name from the list box
```

- Whenever the database executes a query, it first computes a strategy of how to do it efficiently. By preparing the query and reusing it, you ensure that the planning step is done only once.



## Prepared Statements

```
SELECT Books.Price, Books.Title
FROM Books, Publishers
WHERE Books.Publisher_Id = Publishers.Publisher_Id
AND Publishers.Name = the name from the list box
```

```
String publisherQuery
    = "SELECT Books.Price, Books.Title"
    + " FROM Books, Publishers"
    + " WHERE Books.Publisher_Id = Publishers.Publisher_Id AND
Publishers.Name = ?";
PreparedStatement stat = conn.prepareStatement(publisherQuery);
stat.setString(1, Name_Of_Publisher);
```



## Prepared Statements

- A PreparedStatement object becomes invalid after the associated Connection object is closed.
- Many databases automatically cache prepared statements. If the same query is prepared twice, the database simply reuses the query strategy.
- Therefore, the user doesn't need to worry about the overhead of calling prepareStatement.



## Multiple Results



## Multiple Results

- It is possible for a query to return multiple results.
- This can happen when executing a stored procedure, or with databases that also allow submission of multiple SELECT statements in a single query.
- Here is how you retrieve all result sets:
  1. Use the execute method to execute the SQL statement.
  2. Retrieve the first result or update count.
  3. Repeatedly call the `getMoreResults` method to move on to the next result set.
  4. Finish when there are no more result sets or update counts.



## Multiple Results

- It is possible for a query to return multiple results.
- This can happen when executing a stored procedure, or with databases that also allow submission of multiple SELECT statements in a single query.
- Here is how you retrieve all result sets:

```
SELECT authorID, firstName, lastName  
  FROM authors;  
SELECT authorID, firstName  
  FROM authors;  
SELECT authorID  
  FROM authors
```



## Multiple Results

- It is possible to execute a command that returns multiple results.
- This can be done by using a `while` loop with a `do` statement.
- Here is how to do it:

```
boolean isResult = stat.execute(command);
boolean done = false;
while (!done)
{
    if (isResult)
    {
        ResultSet result = stat.getResultSet();
        do something with result
    }
    else
    {
        int updateCount = stat.getUpdateCount();
```





## Scrollable Result Sets



## Scrollable Result Sets

- You usually want the user to be able to move both forward and backward in the result set.
- In a scrollable result, you can move forward and backward through a result set and even jump to any position.

```
while (resultSet.next()) {  
    for (int i = 1; i <= numberOfColumns; i++) {  
        System.out.printf("%-8s\t", resultSet.getObject(i));  
    }  
    System.out.println();  
}
```

## Scrollable Result Sets

- By default, result sets are not scrollable or updatable.
- To obtain scrollable result sets from your queries, you must obtain a different `Statement` object with the method.

```
Statement stat = conn.createStatement(type, concurrency);
```

```
PreparedStatement stat = conn.prepareStatement(command, type, concurrency);
```



## Scrollable Result Sets

- By default, result sets are not scrollable or updatable.
- To obtain scrollable result sets from your queries, you must obtain a different Statement object with the method.

| Value                   | Explanation                                                         |
|-------------------------|---------------------------------------------------------------------|
| TYPE_FORWARD_ONLY       | The result set is not scrollable (default).                         |
| TYPE_SCROLL_INSENSITIVE | The result set is scrollable but not sensitive to database changes. |
| TYPE_SCROLL_SENSITIVE   | The result set is scrollable and sensitive to database changes.     |

Result Set Type Value

## Scrollable Result Sets

- By default, result sets are not scrollable or updatable.
- To obtain scrollable result sets from your queries, you must obtain a different Statement object with the method.

| Value            | Explanation                                                     |
|------------------|-----------------------------------------------------------------|
| CONCUR_READ_ONLY | The result set cannot be used to update the database (default). |
| CONCUR_UPDATABLE | The result set can be used to update the database.              |

ResultSet Concurrency Values

## Scrollable Result Sets

- `if (rs.previous()) ...` To scroll backward
- `rs.relative(n) ....`

If `n` is positive, the cursor moves forward. If `n` is negative, it moves backward. If `n` is zero, the call has no effect.

- `rs.absolute(n) ...` Set the cursor to a particular row number



## Scrollable Result Sets

`if (rs.previous()) ...` To scroll backward  
`rs.relative(n) ....`

If `n` is positive, the cursor moves forward.

If `n` is negative, it moves backward.

If `n` is zero, the call has no effect.

`rs.absolute(n) ...` Set the cursor to a particular row number

- The convenience methods **first**, **last**, **beforeFirst**, and **afterLast** move the cursor to the first, to the last, before the first, or after the last position.
- Finally, the methods **isFirst**, **isLast**, **isBeforeFirst**, and **isAfterLast** test whether the cursor is at one of these special positions

## ○ Updatable Result Sets





## Updatable Result Sets

- Once users see the contents of a result set displayed, they may be tempted to edit it.
- In an Updatable Result Set, you can programmatically update entries so that the database is automatically updated.



## Updatable Result Sets

- If you want to edit the data in the result set and have the changes automatically reflected in the database, create an updatable result set.
- Updatable result sets don't have to be scrollable, but if you present data to a user for editing, you usually want to allow scrolling as well.

```
Statement stat = conn.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE) ;
```

The result sets returned by a call to `executeQuery` are then updatable.

## Updatable Result Sets

```
String query = "SELECT * FROM Books";
ResultSet rs = stat.executeQuery(query);
while (rs.next())
{
    if (. . .)
    {
        double increase = . . .;
        double price = rs.getDouble("Price");
        rs.updateDouble("Price", price + increase);
        rs.updateRow(); // make sure to call updateRow after
updating fields
    }
}
```



## Row Sets



## Row Sets

- Scrollable result sets are powerful, but they have a major drawback.
- **You need to keep the database connection open during the entire user interaction.**
- However, a user can walk away from the computer for a long time, leaving the connection occupied.
- That is not good — **database connections are scarce resources. In this situation, use a row set.**
- The RowSet interface extends the ResultSet interface, but row sets don't have to be tied to a database connection.



# Constructing Row Sets

- The `javax.sql.rowset` package provides the following interfaces that extend the `RowSet` interface:
  - A `CachedRowSet` allows disconnected operation.
  - A `WebRowSet` is a cached row set that can be saved to an XML file. The XML file can be moved to another tier of a web application where it is opened by another `WebRowSet` object.
  - The `FilteredRowSet` and `JoinRowSet` interfaces support lightweight operations on row sets that are equivalent to SQL `SELECT` and `JOIN` operations. These operations are carried out on the data stored in row sets, without having to make a database connection.
  - A `JdbcRowSet` is a thin wrapper around a `ResultSet`. It adds useful methods from the `RowSet` interface.



# Part V

## Wrap-up – What's Next



## Summary





# Summary

In this Hands-on Tutorial lecture:

- We were introduced to the Apache Derby Java RDBMS:
  - Installation
  - Configuration
  - Manipulation
- Performed in-class exercises.



Next Lecture



University of Windsor

## Next Week Lectures

- Last lectures for the semester:
  - Networking in Java
  - Java Microservices



# Part VI

## Training and resources



## ○ Apache Derby



# ● Apache Derby

- <https://db.apache.org/derby/>
- [Apache Derby Tutorial – Eclipse Platform](#)
- <https://www.baeldung.com/java-apache-derby>
- [Apache Derby Database – Tutorial](#)
- Apache Derby alternatives:
  - H2: [Maven Repository: com.h2database » h2](#)
  - HSQLDB: [Maven Repository: org.hsqldb » hsqldb](#)





## Tools



# Tools

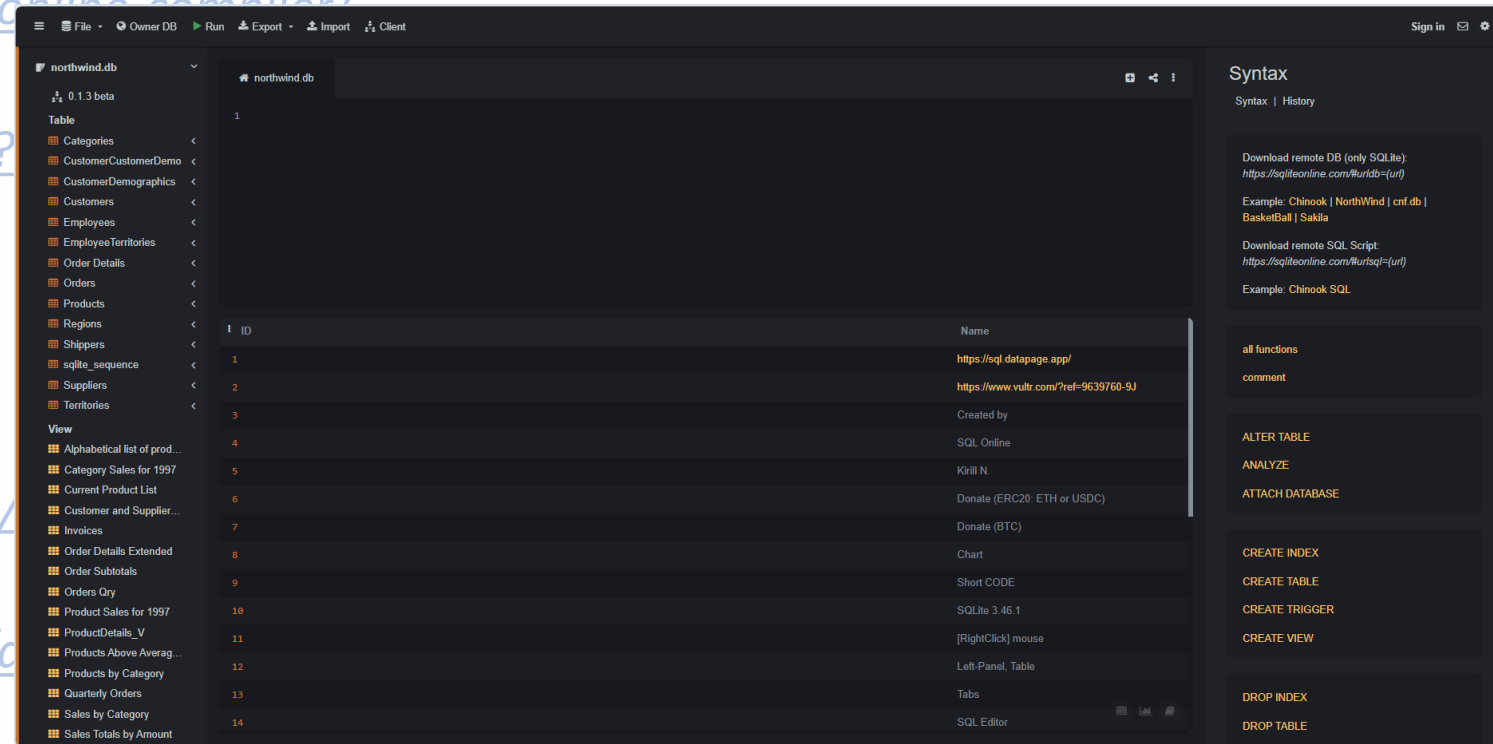
- <https://www.essentialsql.com/sql-tutorial-for-sql-server/>
- [https://www.w3schools.com/sql/sql\\_editor.asp](https://www.w3schools.com/sql/sql_editor.asp)
- <https://www.programiz.com/sql/online-compiler/>
- <https://sqliteonline.com/>
- <https://livesql.oracle.com/apex/f?p=590:1000>
- Code editors:
  - Visual Studio Code / VS Codium
  - Eclipse IDE
  - Notepad++, etc.
- DBeaver: <https://dbeaver.io/>
- SqlDBM: <https://app.sqldbm.com/>
- <https://dbmstools.com/>
- Oracle: <https://www.oracle.com/database/sqldeveloper/technologies/sql-data-modeler/>





# Tools

- <https://www.essentialsql.com/sql-tutorial-for-sql-server/>
- [https://www.w3schools.com/sql/sql\\_editor.asp](https://www.w3schools.com/sql/sql_editor.asp)
- <https://www.programiz.com/sql/online-compiler/>
- <https://sqliteonline.com/>
- <https://livesql.oracle.com/apex/f?>
- *Visual Studio Code*
- *Notepad++*
- *DBeaver*: <https://dbeaver.io/>
- *SqlDBM*: <https://app.sqldbm.com/>
- <https://dbmstools.com/>
- *Oracle*: <https://www.oracle.com/c>



## ○ Training resources



# Training Resources

- Datacamp: <https://app.datacamp.com/learn/courses/introduction-to-sql>
- <https://www.essentialsql.com/>
- <https://www.w3schools.com/sql/>
- <https://www.khanacademy.org/computing/computer-programming/sql/sql-basics/v/welcome-to-sql>
- <https://www.learnsqlonline.org/>
- <https://sqlbolt.com/>
- [https://sqlzoo.net/wiki/SQL\\_Tutorial](https://sqlzoo.net/wiki/SQL_Tutorial)
- <https://www.codecademy.com/resources/docs/sql>
- <https://www.freecodecamp.org/>
- ...



Thank you!  
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

[Andreas.Maniatis@UWindsor.ca](mailto:Andreas.Maniatis@UWindsor.ca)

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor